


video 21
26 Sept 2023

Samarth Sir

24 Oct, 2024

Q guess the o/p :-

```
let marks = {  
  dsa : 55,  
  web : 85,  
  ml : 67,  
  totalMarks : function(fine){  
    return 55 + 85 + 67 - fine  
  }  
}  
  
console.log(marks);  
console.log(marks.totalMarks(50));
```

argument is
passed

o/p →



```
{dsa: 55, web: 85, ml: 67, totalMarks: f} app.js:12  
157 app.js:13  
>
```

Q

```
let marks = {  
  dsa : 55,  
  web : 85,  
  ml : 67,  
  totalMarks : function(fine){  
    return this.dsa + this.web + this.ml - fine  
  }  
}  
  
console.log(marks);  
console.log(marks.totalMarks(50));
```

Same o/p as of last question.

Here we used this keyword.

String methods :-

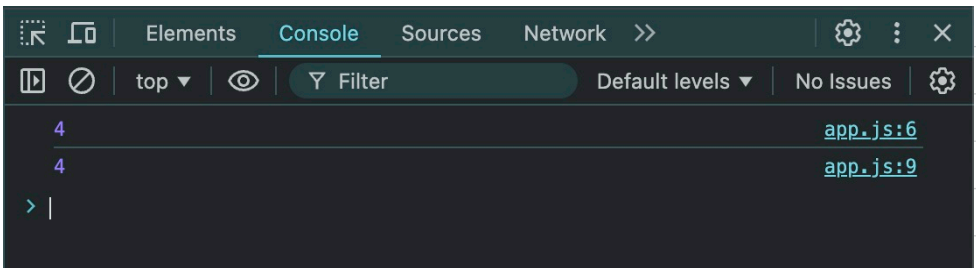
→ ham dekhenge ki string ke kounse already available methods hai.

① • length :- → yaha call nahi karna padega,

it is a property.

```
let str = "Ma V";  
  
let len = str.length;  
console.log(len);  
  
// or  
console.log(str.length);
```

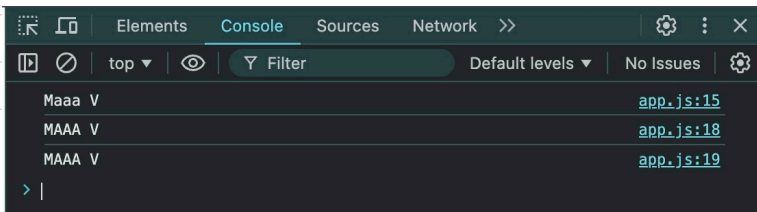
length mei space count hoga.



The screenshot shows the browser's developer console with the 'Console' tab selected. It displays two log entries, both with the value '4'. The first log entry is from 'app.js:6' and the second is from 'app.js:9'. The prompt '> |' is visible at the bottom left of the console area.

② • toUpperCase() → it is a method, ise call karna padega.

```
let str = "Maaa V";  
console.log(str);  
  
let ans = str.toUpperCase();  
console.log(str.toUpperCase());  
console.log(ans);
```

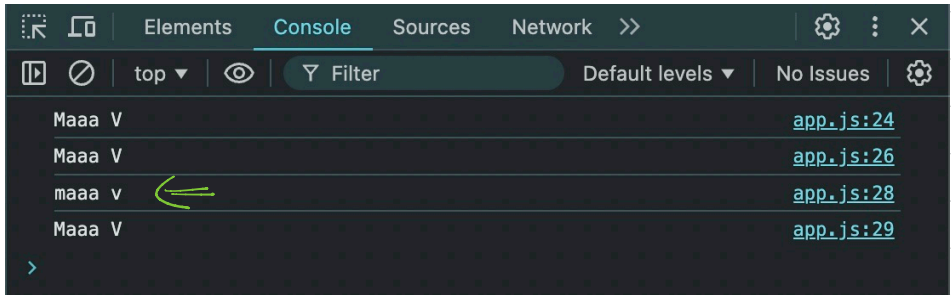


The screenshot shows the browser's developer console with the 'Console' tab selected. It displays three log entries: 'Maaa V' from 'app.js:15', 'MAAA V' from 'app.js:18', and 'MAAA V' from 'app.js:19'. The prompt '> |' is visible at the bottom left of the console area.

③

toLowerCase()

```
let str = "Maaa V";  
console.log(str);  
  
console.log(str);  
let ans = str.toLowerCase();  
console.log(ans);  
console.log(str);
```



Q

"__ samarth" or "samarth"

Space

Kya yedono same hai?

Ans Nahi.

Mam ko user ne apne email address mei space daal diya, how will you handle it?

we will use trim method.

4. Trim method :-

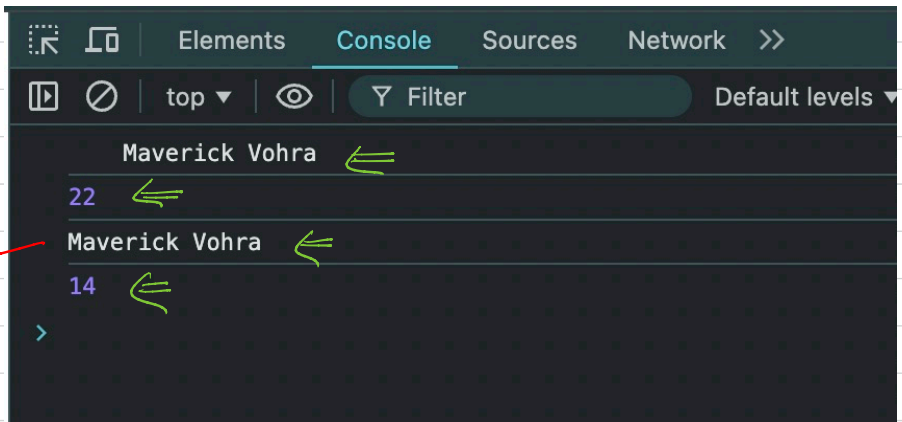
```
let str = "    Maverick Vohra    ";
```

```
console.log(str);  
console.log(str.length);
```

→ l1

```
let trimmedStr = (str.trim());  
console.log(trimmedStr);  
console.log(trimmedStr.length);
```

→ l2



→ aage piche wale space delete ho gye, beech wala reh gya. Beech wala space nahi hatega.

l1 and l2 mei difference dekho,
entara spaces tohin ho gye.

5) includes() :- just observe the output and
code on the next page, you will understand
how it works.
→ ye true ya false hi dega.

```

let str = "    Maverick Vohra    ";
let ans = str.includes("M");
let ans1 = str.includes("m");
let ans2 = str.includes("Maverick");
let ans3 = str.includes(" Mave" );
let ans4 = str.includes(" Mave " );
let ans5 = str.includes("");
let ans6 = str.includes(" ");
console.log(ans);
console.log(ans1);
console.log(ans2);
console.log(ans3);
console.log(ans4);
console.log(ans5);
console.log(ans6);

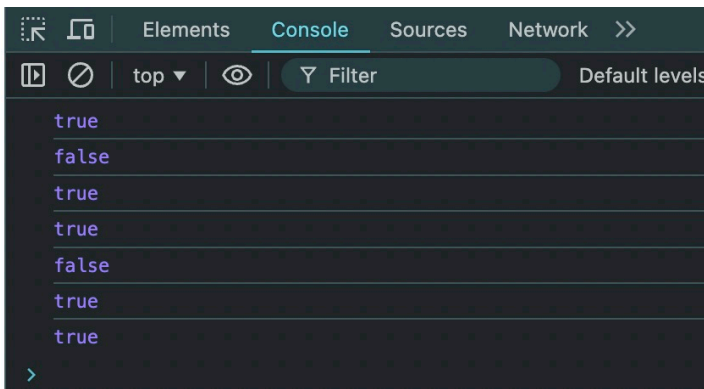
```

case sensitive,
hence false

empty string par
true kyu aaya?



→ The empty string
can be found before
the first character,
between any two
characters, and after
the last character.
Therefore, it's always
true.



`includes()` method determines whether an array /
string includes a certain value among its
entries. It either returns true or false.

(38:08)

Method Chaining :-

```
let str = "    Maverick Vohra    ";  
let ans1 = str.trim();  
console.log(ans1);  
let ans2 = str.trim().length;  
console.log(ans2);  
let ans3 = str.trim().toUpperCase().length;  
console.log(ans3);  
let ans4 = str.trim().toUpperCase();  
console.log(ans4);
```

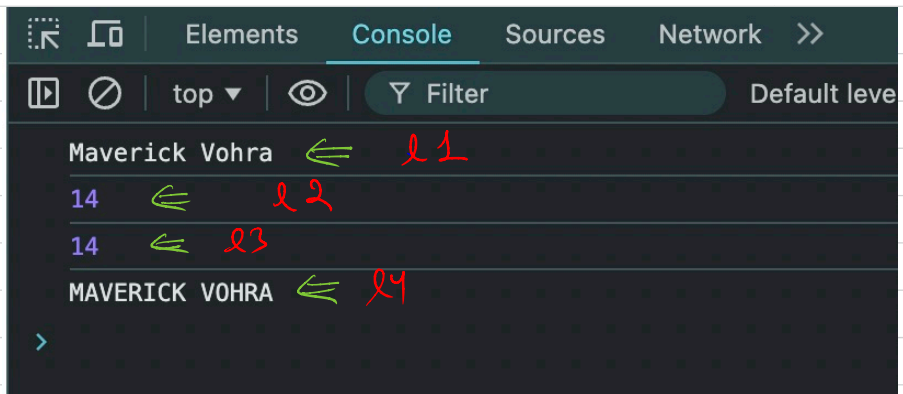
l1

l2

l3

l4

Hamne ek hi line mei multiple methods use kar liye.



Number methods :-

• length will not work for numbers.

```
let ans = 333;  
console.log(ans.length);
```

→ it will give undefined.

The reason `ans.length` gives `undefined` is because `333` is a **number**, and numbers in JavaScript do not have a `length` property. The `length` property is primarily available on **strings** and **arrays**, but not on numbers.

```
Elements Console Sources Network >>
top Filter Default levels
undefined
> |
```

`console.log ( ans.length() );`

→ ye error deqa. error kaheya ki
`ans.length` is not a function.

Math → it is an object.

agar mai `console.log (Math)` karunga. I will be
able to see its properties.

```
Elements Console Sources Network >>
top Filter Default levels No Issues
▼ Math {abs: f, acos: f, acosh: f, asin: f, asinh: f, ...} app.js:10
  E: 2.718281828459045
  LN2: 0.6931471805599453
  LN10: 2.302585092994046
  LOG2E: 1.4426950408889634
  LOG10E: 0.4342944819032518
  PI: 3.141592653589793
  SQRT1_2: 0.7071067811865476
  SQRT2: 1.4142135623730951
  ▶ abs: f abs()
  ▶ acos: f acos()
  ▶ acosh: f acosh()
  ▶ asin: f asin()
  ▶ asinh: f asinh()
  ▶ atan: f atan()
  ▶ atan2: f atan2()
  ▶ atanh: f atanh()
  ▶ cbrrt: f cbrrt()
  ▶ ceil: f ceil()
  ▶ clz32: f clz32()
  ▶ cos: f cos()
  ▶ cosh: f cosh()
  ▶ exp: f exp()
  ▶ expm1: f expm1()
  ▶ floor: f floor()
  ▶ fround: f fround()
  ▶ hypot: f hypot()
  ▶ imul: f imul()
  ▶ log: f log()
  ▶ log1p: f log1p()
  ▶ log2: f log2()
  ▶ log10: f log10()
  ▶ max: f max()
  ▶ min: f min()
  ▶ pow: f pow()
  ▶ random: f random()
  ▶ round: f round()
```

properties
and
methods ←

⇒ Since Math
is an object,
If I want to
use any of
its property
& method, I
will use dot
notation

```
EXPLORER    ...  ne    JS app.js  methods    JS app.js  string methods    JS
▼ LEC 21
  > methods
  ▼ number methods
    JS app.js
    <> index.html
    > string methods

number methods > JS app.js
1  // 1..
2  💡
3  // let ans = 333;
4  // console.log(ans.length);
5
6  // console.log(ans.length());
7
8  // 2..
9
10 console.log(Math);
```

Lec 21 ⇒ 50:00

0.69 console.log (Math.LN2);
2.30 console.log (Math.LN10);
3.14 console.log (Math.PI); } ⇒ this is how we
can use Math
object.

```
console.log(Math.ceil(4.3)); //5
console.log(Math.floor(4.3)); //4

console.log(Math.round(4.3)); //4
console.log(Math.round(4.8)); //5
console.log(Math.round(4.5)); //5

console.log(Math.pow(2,5)); // 32
console.log(Math.pow(2,13)); // 8192
```

4.3 ka ceil is 5 (+1 karke decimal
hata do)

4.3 ka floor is 4 (decimal hata do)

4.3 ka round → 4

4.5 " " → 5

4.9 " " → 5

} decimal value Sya
5 se jyada ho, then
plus 1 kar do.

$\text{Math.pow}(2, 5) \Rightarrow 2^5 \Rightarrow 32$

$\text{Math.random}()$ $\Rightarrow$ it gives 0 or numbers between 0 and 1.

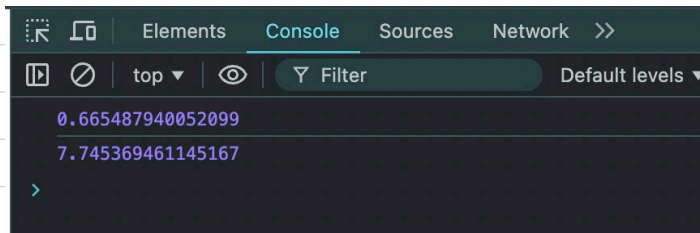
Range $[0, 1)$

o/p mei 0 aa sakta hai but 1 nahi.

0.92

it generates numbers randomly

```
console.log(Math.random());  
console.log(Math.random() * 10);
```



$\rightarrow$ otp generate karne mei kaam aayega.

Q mujhe 6 digits ka otp chahiye:-

```
console.log(Math.floor(Math.random()*1000000));
```

$\rightarrow$ this will generate 6 random 6 digits numbers.

0 mujhe 0 to 9 ke beech ke random numbers chahiye without decimal :-

console.log (Math.floor (Math.random() * 10));

0 to 9 $\rightarrow$ * 10 $\rightarrow$ Range $\rightarrow 9 - 0 \rightarrow 9 + 1 = 10$

0 to 99 $\rightarrow$ * 100

0 to 999 $\rightarrow$ * 1000

$\downarrow$
multiplication
factor

what if I want to get numbers b/w 5 to 99.

$\rightarrow$ Range = $99 - 0 = 99 + 1 \rightarrow 100$

$\downarrow$
multiplication
factor

5 to 99

$\hookrightarrow$

iss range ke numbers chahiyete
multiplication factor nikalo

$99 - 5 \rightarrow 94 + 1 \Rightarrow 95$

$\hookrightarrow$

$\downarrow$

Range

$\downarrow$
multiplication
factor

But humko minimum value jo chahiye vo 5
honi chahiye. Isliye +5 kar do. Jo bhi number
aayega usko 5 se aage bada do.

Let's take one more example to understand it better :-

Maamlo mujhe 5 to 9 ke beech ke numbers chahiye

Step 1 :-

$\text{Math.floor}(\text{Math.random()} * 10)$

↓

0.123

↓

1.23

↓

1

Range is 0 to 9.

Step 2 :-

Ab agar mai har number mei 5 add kar du, let's say

$\text{Math.floor}(\text{Math.random()} * 5 + 5);$

↓

0.091

→ Multiplication factor

↓

0.455

= $9 - 5 + 1$

→ 5

↓

4.55

↓

4

Range aage khisak gyi 5 add karne se.

Steps ①

$\text{Math.floor}(\underbrace{\text{Math.random()} * 5}_{0.999} + 5);$

0.999

4.995

9.995

9

```
console.log( Math.floor( (Math.random() * 95) + 5 ) );
```

```
console.log( Math.floor( (Math.random() * 5) + 5 ) );
```

Practice questions :-

① $17 - 33 \Rightarrow 33 - 17 \Rightarrow 16 + 1 \Rightarrow 17$

$(* 17 + 17)$

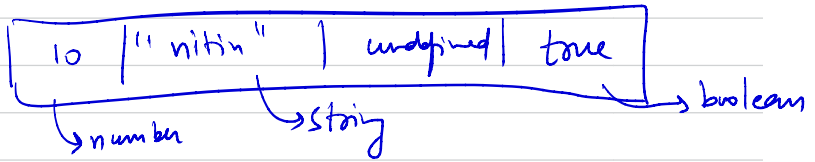
② $0 - 19 \Rightarrow (19 - 0) + 1 \Rightarrow 20 + 0$

③ $13 - 40 \Rightarrow 40 - 13 \Rightarrow 27 + 1 \Rightarrow 28$

$(* 28 + 13)$

Array :- ordered data structure, heterogeneous.

aap ek hi array mei different datatypes
ki values store kar sakte ho.



Syntax: `let nitin = [ ];`

`let arr = ["sam", 5, "hi", undefined,
null];`

```
let arr = ['nitin', 9, undefined, null];
```

```
> arr
< ▶ (4) ['nitin', 9, undefined, null]
>
```

`arr[1] → 9`

`arr[2] → undefined`

```
Elements Console Sources Network >>
top Filter Default levels No Issues
> arr
< ▶ (4) ['nitin', 9, undefined, null]
> arr[1] ↵
< 9
> arr[34] ↵
< undefined
>
```

`arr[34]` → `undefined` aaya kyunki humne sirf 3rd index tak value di hai.

`arr[1] → 10;`
you can also update an array

```
Elements Console Sources Network >>
top Filter Default levels
> arr
< ▶ (4) ['nitin', 9, undefined, null]
> arr[1]
< 9
> arr[34]
< undefined
> arr[1] = 199; ↵
< 199
> arr ↵
< ▶ (4) ['nitin', 199, undefined, null]
>
```

you can also add/create object inside array.

```
Elements Console Sources Network >>
top [Filter] Default levels ▾

> arr
< ▶ (4) ['nitin', 9, undefined, null]

> arr[1]
< 9

> arr[34]
< undefined

> arr[1] = 199;
< 199

> arr
< ▶ (4) ['nitin', 199, undefined, null]

> arr[0] = {obj: 'hi there'}
< ▶ {obj: 'hi there'}

> arr
< ▾ (4) [{...}, 199, undefined, null] i ←
  ▶ 0: {obj: 'hi there'} ←
    1: 199
    2: undefined
    3: null
    length: 4
  ▶ [[Prototype]]: Array(0)

>
```

2D array :- array inside array.

```
let arr2 = [10, "samarth", true, null, [1000, [200, 300, 400], 3000], 100];
```

```
console.log(arr2[4]);
console.log(arr2[4][1]);
console.log(arr2[4][1][1]);
```

```
arr2[4][1][1] = "samarth";
console.log(arr2[4][1][1]);
```



```
Elements Console Sources Network >>
Create live expression
[1000, Array(3), 3000]
[200, 300, 400]
300
samarth
>
```

arr2 [120] = "I am fine";
↪ value assign ho jayegi. JS is very flexible.

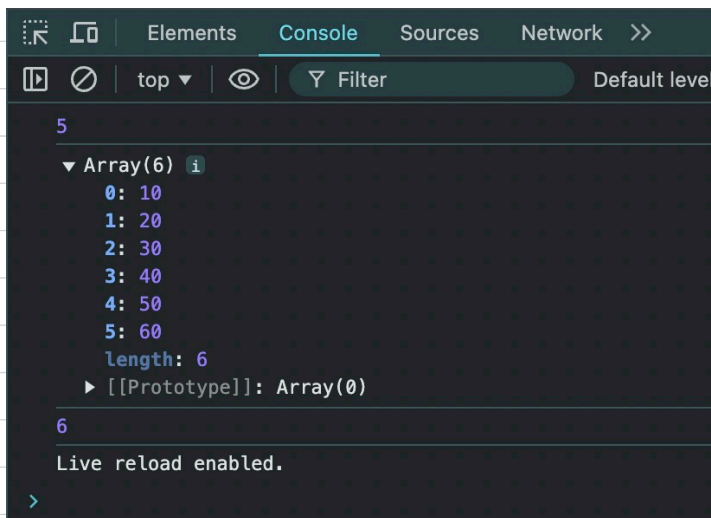
```
Elements Console Sources Network >>
[1000, Array(3), 3000] app.js:16
[200, 300, 400] app.js:17
300 app.js:18
samarth app.js:21
> arr2[4500] = 'param brahm param dham'
< 'param brahm param dham'
> arr2
< (4501) [10, 'samarth', true, null, Array(3), 100, empty × 4494, 'param br
  ahm param dham'] i
  0: 10
  1: "samarth"
  2: true
  3: null
  4: (3) [1000, Array(3), 3000]
  5: 100
  4500: "param brahm param dham"
  length: 4501
  [[Prototype]]: Array(0)
>
```

Array Methods :-

① `.length`

② `.push()` → add elements to the last of an array.

```
let arr = [10,20,30,40,50];  
console.log(arr.length);  
arr.push(60);  
console.log(arr)  
console.log(arr.length);
```

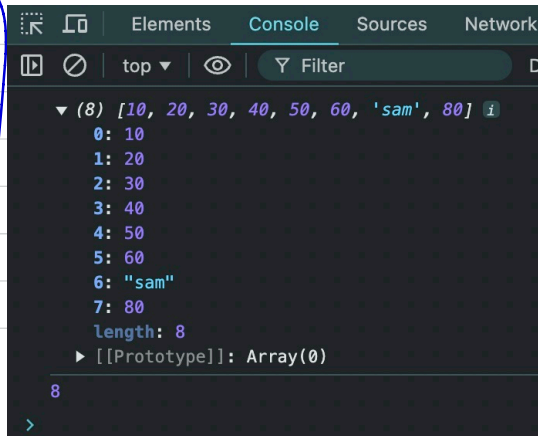


The screenshot shows the Chrome DevTools Console with the 'Console' tab selected. The output is as follows:

```
5  
▼ Array(6) i  
  0: 10  
  1: 20  
  2: 30  
  3: 40  
  4: 50  
  5: 60  
  length: 6  
  ► [[Prototype]]: Array(0)  
6  
Live reload enabled.  
>
```

A blue arrow points from the `console.log(arr)` line in the code block above to the array object in the console output.

```
let arr = [10,20,30,40,50];  
arr.push(60 , "sam" , 80);  
console.log(arr)  
console.log(arr.length);
```



The screenshot shows the Chrome DevTools Console with the 'Console' tab selected. The output is as follows:

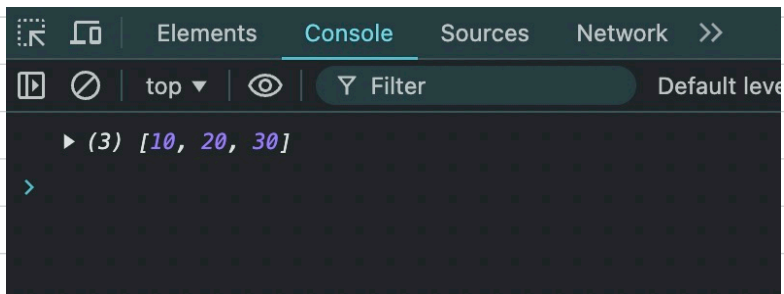
```
▼ (8) [10, 20, 30, 40, 50, 60, 'sam', 80] i  
  0: 10  
  1: 20  
  2: 30  
  3: 40  
  4: 50  
  5: 60  
  6: "sam"  
  7: 80  
  length: 8  
  ► [[Prototype]]: Array(0)  
8  
>
```

A blue arrow points from the `console.log(arr)` line in the code block above to the array object in the console output.

③ .pop() → remove last element of array

→ you don't need to give any argument in it because it will only remove last element even if you pass an argument.

```
let arr = [10,20,30,40,50];  
arr.pop();  
arr.pop(20);  
console.log(arr);
```



→ last ke dono elements hatti gye.

```
let arr = [10,20,30,40,50];  
let ans = arr.pop(); //last value pop hui hai  
let ans2 = arr.push(400); //length  
  
console.log(ans)  
console.log(ans2)
```



```
50 ⇒ l1
5 ⇒ l2
> |
```

l1 ⇒ jo element pop hua wahi element return ho raha hai.

l2 ⇒ push hone ke baad array ki ^{new} length return ho rahi hai.